# TEMA 7 – Funciones de Orden Superior

> **Objetivo del tema**
>
> En este tema aprenderemos sobre las Funciones de Orden Superior

## 7.1 Funciones de Orden Superior

Consulta la presentación **UD07 - Funciones de Orden Superior**.

Una **Función de Orden Superior** (*Higher-Order Function* o **HOF**) es una función que cumple al menos una de estas dos condiciones:

1. **Recibe** una o más funciones como argumentos (llamadas *callbacks*).
2. **Devuelve** una función como resultado.

Esto es posible en JavaScript las funciones se pueden guardar en variables, pasar por parámetro o retornar exactamente igual que un número, un texto o un objeto.

Las **HOF** no aportan nada nuevo a la programación que no existiese ya. Es decir: tú puedes seguir programando sin utilizar nunca este tipo de funciones. No obstante, ofrecen algunas ventajas significativas:

- Código más legible.
- Menos errores.
- Más fácil de probar y mantener.

Su principal utilidad en JavaScript es manejar **Arrays**; aunque obviamente, pueden utilizarse para más cosas.

A modo de ejemplo simple, este código coge un array de números, multiplica cada elemento por 2, y lo guarda en un nuevo array. No se están usando HOF.

```
const array = [1, 4, 9, 16];
const mapped = [];
for (let i = 0; i < array.length; i++)
{
  mapped.push(array[i] * 2);
}
```

Estamos operando con funciones 'tradicionales'. En cambio, mira este código:

```
const array = [1, 4, 9, 16];
const mapped = array.map((x) => x * 2);
```

En este caso, sí estamos usando HOF. El método **map()** es la HOF, y la función de flecha **(x) => x * 2** es el ***callback*** que se le pasa como argumento para que lo aplique a cada elemento del array. Como ves, es mucho más cómodo y legible utilizar estas funciones.

## 7.2 ¿Por qué existen las HOF?

Porque obviamente, son una mejora respecto a lo que teníamos antes. También refleja un intento de simplificar el recorrido de datos y avanzar hacia la programación funcional. Para entendernos:

Así es como se recorría un Array originalmente en JavaScript:

```
for (let i = 0; i < numeros.length; i++) {
  console.log(numeros[i]); // acceso manual al índice
}
```

¿Por qué tenemos que gestionar manualmente 'i'? Esto es una fuente de potenciales errores. Además, en la mayoría de las ocasiones, no nos interesa una posición en concreto, ni nos interesa saber en qué posición hay un número 7: simplemente si lo hay o no lo hay.

Por este motivo, se inventó el forEach (HOF):

```
numeros.forEach(numero => console.log(numero));
```

Así, en lugar de gestionar el índice y la iteración manualmente, le delegas el control al método del array pasándole una función ***callback***. No obstante, este tipo de funciones no permiten break, continue o return; por lo que finalmente se introdujo la sintaxis siguiente:

```
for (const numero of numeros) {
  console.log(numero); // más claro, sin índice
}
```

## 7.3 HOF en los Arrays – map()

El método **map()** es un método de los arrays en JavaScript que pertenece al grupo de las **HOF**. Sirve para transformar cada elemento de un array y devuelve un nuevo array con los resultados de esa transformación, manteniendo siempre la misma longitud que el array original.

Luego, esta función tiene dos partes:

1) 'Hace algo' con cada elemento del Array.

2) Devuelve un nuevo array, que es el original al que le 'ha hecho algo'. Ç

Ese 'algo' que se ejecuta en cada elemento es una función a la que llamamos ***callback.***

Mira estos dos ejemplos:

```
const numeros = [1, 2, 3];
const dobles = numeros.map(function(numero) {
  return numero * 2;
});


const nombres = ["ana", "josu", "peio"];
const capitalizados = nombres.map(nombre =>
  nombre.toUpperCase()
);
```

Ambas anotaciones son válidas, y esencialmente lo mismo: map() está recibiendo una función (callback) como parámetro de entrada, y lo aplica sobre cada elemento del Array. El segundo ejemplo se corresponde a una **Arrow Function** (función flecha) y se recomienda usarlos siempre que sea posible.

Obviamente, es posible utilizar todo lo aprendido con Arrays de objetos:

```
const usuarios = [
  { id: 1, nombre: "Ana", edad: 20 },
  { id: 2, nombre: "Luis", edad: 25 }
];

const soloNombres = usuarios.map(usuario => usuario.nombre);

console.log(soloNombres); // ["Ana", "Luis"]
```

## 7.4 HOF en los Arrays – filter()

El método **filter()** es un método de los arrays que permite seleccionar únicamente los elementos que cumplan una condición. Devuelve dichos elementos en forma de un nuevo Array.

```
const numeros = [2, 5, 8, 1, 10];
const mayoresQueCinco = numeros.filter(numero => numero > 5);

console.log(mayoresQueCinco); // [8, 10]
```

## 7.5 HOF en los Arrays – find()

El método **find()** es un método de los arrays que devuelve el primer elemento que cumpla una condición; o undefined si no lo encuentra.

```
const numeros = [3, 7, 12, 15];
const primeroMayorQueDiez = numeros.find(numero => numero > 10);

console.log(primeroMayorQueDiez); // 12
```

## 7.6 HOF en los Arrays – some()

El método **some()** es un método de los arrays que comprueba si existe al menos un elemento que cumpla una condición. Devuelve true o false.

```
const usuarios = [
  { nombre: "Ana", conectado: false },
  { nombre: "Luis", conectado: false },
  { nombre: "Marta", conectado: true }
];

const hayConectado = usuarios.some(usuario => usuario.conectado);

console.log(hayConectado); // true
```

## 7.7 HOF en los Arrays – every()

El método **every()** es un método de los arrays que verifica si todos los elementos del array cumplen una condición. Si todos la cumplen, devuelve true.

```
const pedidos = [
  { id: 1, entregado: true },
  { id: 2, entregado: true },
  { id: 3, entregado: false }
];

const todosEntregados = pedidos.every(pedido => pedido.entregado);

console.log(todosEntregados); // false
```

## 7.8 HOF en los Arrays – reduce()

El método **reduce()** es un método de los arrays que procesa un array elemento por elemento para "reducirlo" a un **único valor final**. Lo que pasa es que ese valor puede ser un número, un texto, un objeto o incluso otro array.

Ejemplo simple:

```
const numeros = [5, 10, 15];
const total = numeros.reduce((acumulador, numero) =>
  acumulador + numero, 0 // valor inicial = 0
);

console.log(total); // 30
```

En este caso, se está realizando un sumatorio de todos los elementos del Array; pero NO tiene por qué ser siempre así.

El método every () recibe dos parámetros:

1) **Callback:** Se ejecuta por cada elemento, y a su vez necesita de:

    a. **acumulador**: La "caja" donde vas guardando el resultado parcial.
    b. **Elemento Actual**: El elemento del array que se está procesando en ese momento.

2) **El valor Inicial:** El valor con el que empieza el acumulador. Si lo omites, tomará por defecto el primer elemento del array.

Date cuenta que hemos dicho que reduce () no tiene por qué sumar todos los elementos de un Array. Si te fijas en el ejemplo siguiente, verás que podemos usar la función para contar el número de piezas de cada fruta:

```
const frutas = ["manzana", "pera", "manzana", "naranja", "pera", "manzana"];
const contador = frutas.reduce((acc, fruta) => {
  acc[fruta] = (acc[fruta] || 0) + 1;
  return acc;
}, {});

console.log(contador); // { manzana: 3, pera: 2, naranja: 1 }
```

## 7.9 HOF en los Arrays – sort()

El método **sort()** es un método de los arrays que ordena los elementos del array según un criterio. A diferencia de otros métodos anteriores, sort() **MODIFICA** el Array original.

El problema con sort() es que puede inducir a errores:

```
const letras = ["b", "a", "c"];
letras.sort();
console.log(letras); // ["a", "b", "c"]
```

Este código funciona correctamente. No obstante, este otro no:

```
const numeros = [10, 1, 5, 20];
numeros.sort();
console.log(numeros); // [1, 10, 20, 5] orden incorrecto
```

Esto sucede porque JavaScript convierte cada elemento a una cadena de texto (string) y los compara según su orden en el código Unicode/ASCII. La forma de solucionarlo es **pasarle** una función de comparación que le diga cómo debe ordenar los elementos.

En este caso:

```
const numeros = [10, 1, 5, 20];
numeros.sort((a, b) => a - b); // ascendente
console.log(numeros); // [1, 5, 10, 20]
```

Donde:

- **Si a - b es negativo (< 0):** Coloca a antes que b.
- **Si a - b es positivo (> 0):** Coloca b antes que a.
- **Si a - b es cero (=== 0):** Mantiene el orden original entre ambos.

## 7.10 HOF en los Arrays – flat() y flatMap()

Estos métodos son un tanto peculiares. **flat()** se usa para "aplanar" arrays anidados, eliminando niveles de anidación. Por su parte, **flatMap()** es una mezcla entre map() y flat(1): transforma y aplana a la vez. Ambas devuelven un nuevo array, pero NO modifican el Array original.

Un ejemplo con **flat ()**:

```
const numeros = [1, [2, 3], [4, [5, 6]]];

const plano1 = numeros.flat(); // profundidad 1
console.log(plano1); // [1, 2, 3, 4, [5, 6]]

const plano2 = numeros.flat(2);  // profundidad 2
console.log(plano2); // [1, 2, 3, 4, 5, 6]
```

Mientras que con **flatMap()**:

```
const frases = ["hola mundo", "buenos días"];

const palabras = frases.flatMap(frase => frase.split(" "));

console.log(palabras); // ["hola", "mundo", "buenos", "días"]
```

## 7.11 HOF en los Arrays – Métodos Inmutables

Los métodos inmutables son nuevos métodos introducidos en la ES2023.Son versiones **inmutables** de los métodos anteriormente vistos. NO modifican el array original, y devuelven una copia transformada. Útiles para trabajar de forma **segura y predecible** sin efectos secundarios.

Hasta ES2022, muchos métodos como sort(), reverse() o splice() modificaban el array original. Con ES2023 tenemos sus versiones inmutables:

| Nuevo método | Antiguo equivalente | Diferencia |
|---|---|---|
| toSorted() | sort() | no muta |
| toReversed() | reverse() | no muta |
| toSpliced() | splice() | no muta |
| with() | (nuevo) | copia un array cambiando 1 elemento |

A modo de ejemplo:

```
const numeros = [3, 1, 2];
const ordenado = numeros.toSorted((a, b) => a - b);

console.log(ordenado); // [1, 2, 3]
console.log(numeros);  // [3, 1, 2] → intacto
```

Y para with ():

```
const numeros = [10, 20, 30];
const actualizado = numeros.with(1, 99);

console.log(actualizado); // [10, 99, 30]
console.log(numeros);     // [10, 20, 30]
```

## Ejercicio HOF Básicos - I

Dado el array [1, 2, 3, 4], crea un nuevo array pero que contenga el doble en cada posición.

Salida esperada: [2, 4, 6, 8]

## Ejercicio HOF Básicos - II

Dado [5, 8, 13, 21, 34], obtén solo los números pares.

*Salida esperada:* [8, 34]

## Ejercicio HOF Básicos - III

Dado [10, 15, 20, 25], encuentra el primer número mayor que 18.

*Salida esperada:* 20

## Ejercicio HOF Básicos - IV

Comprueba si el array [2, 3, 5, 7, 11] contiene algún número mayor que 10.

*Salida esperada:* true

## Ejercicio HOF Básicos - V

Comprueba si todos los números de [12, 24, 36] son múltiplos de 6.

*Salida esperada:* true

## Ejercicio HOF Básicos - VI

Dado:

```
const usuarios = [
  { nombre: "Ana", activo: true },
  { nombre: "Luis", activo: false },
  { nombre: "Marta", activo: true }
];
```

Obtén un array con los nombres de los usuarios activos.

*Salida esperada:* ["Ana", "Marta"]

## Ejercicio HOF Básicos - VII

Dado:

```
const pedidos = [
  { id: 1, total: 50, entregado: true },
  { id: 2, total: 30, entregado: false },
  { id: 3, total: 70, entregado: true }
];
```

Calcula el total de ventas entregadas usando filter() + reduce().

*Salida esperada:* 120

## Ejercicio HOF Básicos - VIII

Dado:

```
const alumnos = [
  { nombre: "Pedro", nota: 6 },
  { nombre: "Lucía", nota: 9 },
  { nombre: "Carlos", nota: 7 }
];
```

Ordénalos de mayor a menor nota usando toSorted() (versión inmutable).

*Salida esperada:* [Lucía, Carlos, Pedro]

## Ejercicio HOF Básicos - IX

Dado:

```
const grupos = [
  ["ana", "peio"],
  ["luis"],
  ["marta", "jon"]
];
```

Usa flat() para obtener un único array con todos los nombres.

## Ejercicio HOF Avanzados - I

Dado:

const notas = [5, 7, 8, 10, 4, 9];

Usando filter() y reduce():

1. Quédate solo con las notas **aprobadas (≥5)**
2. Súmalas
3. Calcula la **media de aprobados**

*Salida esperada:* 7.8

## Ejercicio HOF Avanzados - II

Dado:

const frutas = ["manzana", "pera", "manzana", "naranja", "pera", "manzana"];

Crea un objeto que indique cuántas veces aparece cada fruta.

*Salida esperada*: { manzana: 3, pera: 2, naranja: 1 }

## Ejercicio HOF Avanzados - III

Dado:

const pedidos = [

{ id: 1, cliente: "Ana", total: 100, entregado: true },

{ id: 2, cliente: "Luis", total: 50, entregado: false },

{ id: 3, cliente: "Ana", total: 75, entregado: true }

];

Calcula cuánto ha gastado en total cada cliente (solo pedidos entregados). Usa filter(), reduce() y devuelve un bjeto tipo: { Ana: 175 }